

Lecture 2

BASICS

CS312

Tatsu H

Logistics

Assignments

- A1 out. Due Midnight Monday

Quiz

- Q1 on Tuesday during class time.
- Just show up – closed book, no cheatsheet etc.
- We will go over answers right after in class.

Office hours

- We will have them for next 2 weeks. See website for when!

Deep learning is now an experimental science

GLU Variants Improve Transformer

Noam Shazeer
Google
noam@google.com

February 14, 2020

4 Conclusions

We have extended the GLU family of layers and proposed their use in Transformer. In a transfer-learn setup, the new variants seem to produce better perplexities for the de-noising objective used in pre-train as well as better results on many downstream language-understanding tasks. These architectures are simple to implement, and have no apparent computational drawbacks. We offer no explanation as to why these architectures seem to work; we attribute their success, as all else, to divine benevolence.

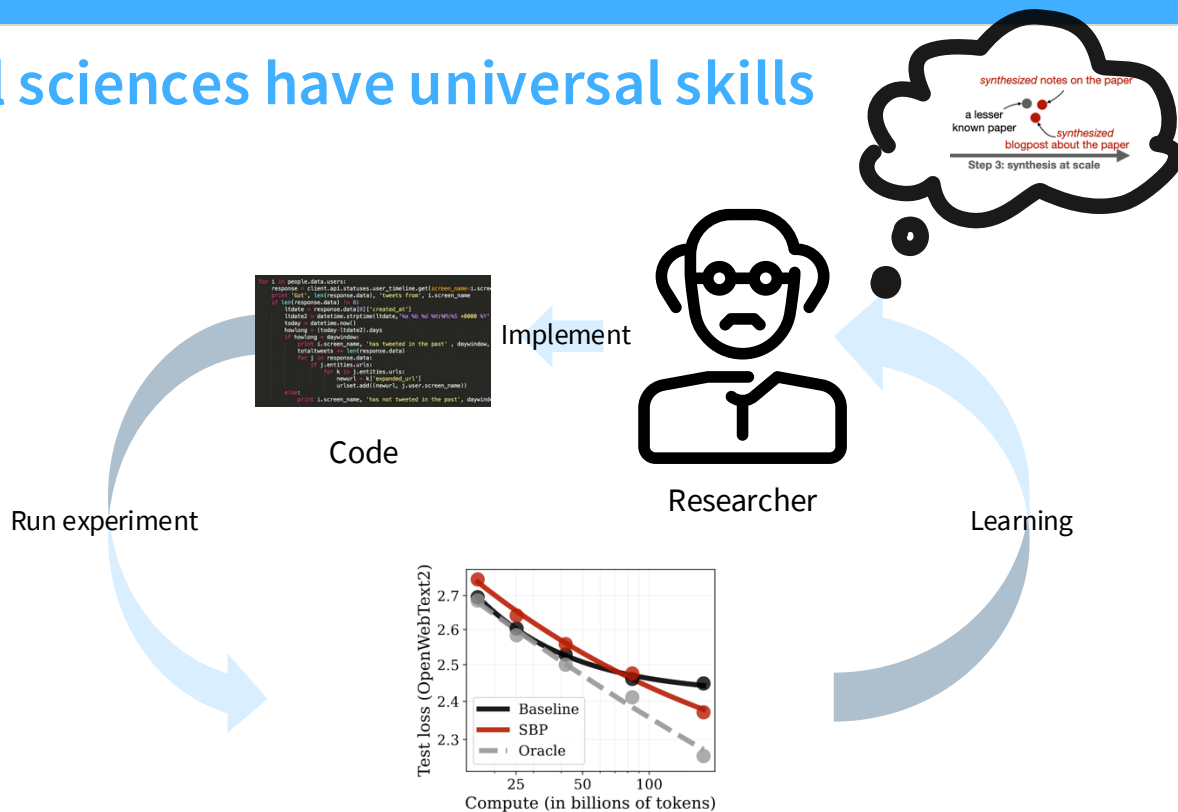
Do Transformer Modifications Transfer Across Implementations and Applications?

Sharan Narang* Hyung Won Chung Yi Tay William Fedus
Thibault Fevry[†] Michael Matena[†] Karishma Malkan[†] Noah Fiedel
Noam Shazeer Zhenzhong Lan[†] Yanqi Zhou Wei Li
Nan Ding Jake Marcus Adam Roberts Colin Raffel

Google Research

Model	Params	Ops	Step/s	Early loss	Final loss	SGLUE
Vanilla Transformer	223M	11.1T	3.50	2.182 ± 0.005	1.838	70.97
GeLU	223M	11.1T	3.58	2.179 ± 0.003	1.838	73.67
Swish	223M	11.1T	3.62	2.186 ± 0.003	1.847	72.03
ELU	223M	11.1T	3.56	2.270 ± 0.007	1.932	65.86
GLU	223M	11.1T	3.59	2.174 ± 0.003	1.814	67.86
GeGLU	223M	11.1T	3.55	2.130 ± 0.006	1.792	74.86
ReGLU	223M	11.1T	3.57	2.145 ± 0.004	1.803	73.40
SeLU	223M	11.1T	3.55	2.315 ± 0.004	1.948	66.13
SwiGLU	223M	11.1T	3.53	2.127 ± 0.003	1.789	74.21
LiGLU	223M	11.1T	3.59	2.149 ± 0.005	1.798	73.97
Sigmoid	223M	11.1T	3.63	2.291 ± 0.019	1.867	71.27
Softplus	223M	11.1T	3.47	2.207 ± 0.011	1.850	70.86
RMS Norm	223M	11.1T	3.68	2.167 ± 0.008	1.821	73.73
Rezero	223M	11.1T	3.51	2.262 ± 0.003	1.939	57.21
Rezero + LayerNorm	223M	11.1T	3.26	2.223 ± 0.006	1.858	68.16
Rezero + RMS Norm	223M	11.1T	3.34	2.221 ± 0.009	1.875	65.14
Fixup	223M	11.1T	2.95	2.382 ± 0.012	2.067	55.71
24 layers, $d_{\text{ff}} = 1536$, $H = 6$	224M	11.1T	3.33	2.200 ± 0.007	1.843	72.55
18 layers, $d_{\text{ff}} = 2048$, $H = 8$	223M	11.1T	3.38	2.185 ± 0.005	1.831	74.74
8 layers, $d_{\text{ff}} = 4608$, $H = 18$	223M	11.1T	3.69	2.190 ± 0.005	1.847	72.17
6 layers, $d_{\text{ff}} = 6144$, $H = 24$	223M	11.1T	3.70	2.201 ± 0.010	1.857	70.28

Experimental sciences have universal skills

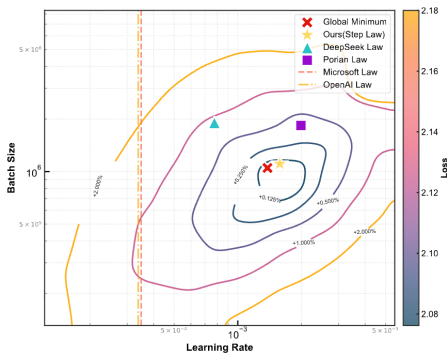


Deep learning is a lot less like CS, a lot more like biology

Tight experiment loops, learning from and designing experiments are very important

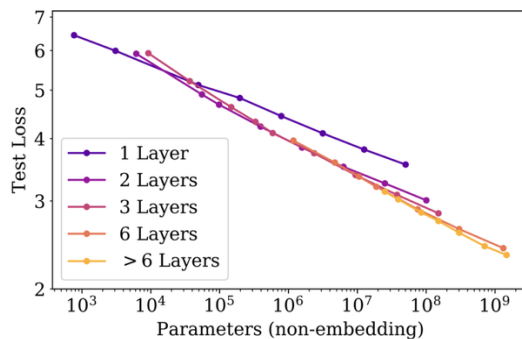
Building good basic skills for experimentation

Optimizing the system



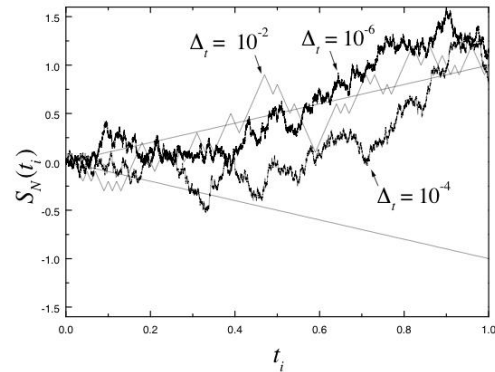
Part 1: Hyperparams

Being efficient



Part 2: Scaling

Measuring the right thing



Part 3: Stochasticity
Part 4: Debugging

Hyperparameters

What's a hyperparameter? Anything we choose by hand (often any *nuisance* parameter)

Optimizer

Which optimizer?

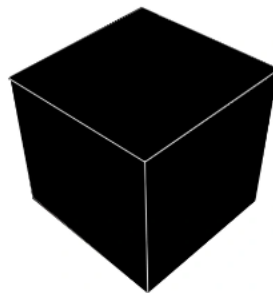
What learning rate?

Architecture

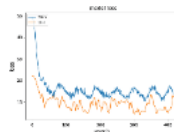
QK norm yes or no?



Deep learning

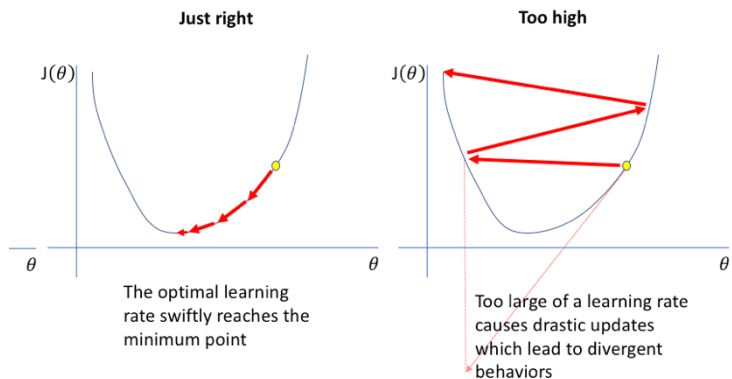


Outcome

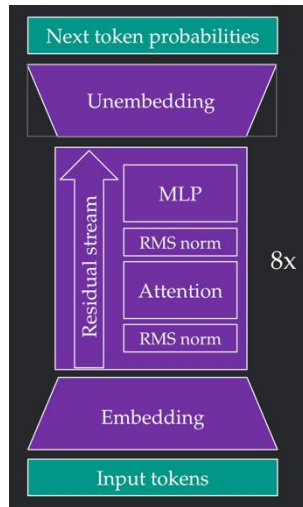


When an experiment 'fails', the only thing we know is that *at least one* setting is baad

Common (and important?) hyperparameters

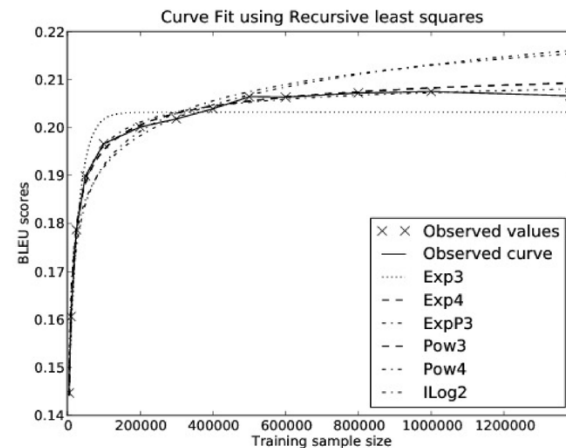


Learning rate



Model size / shape

Learning rate



Number of steps

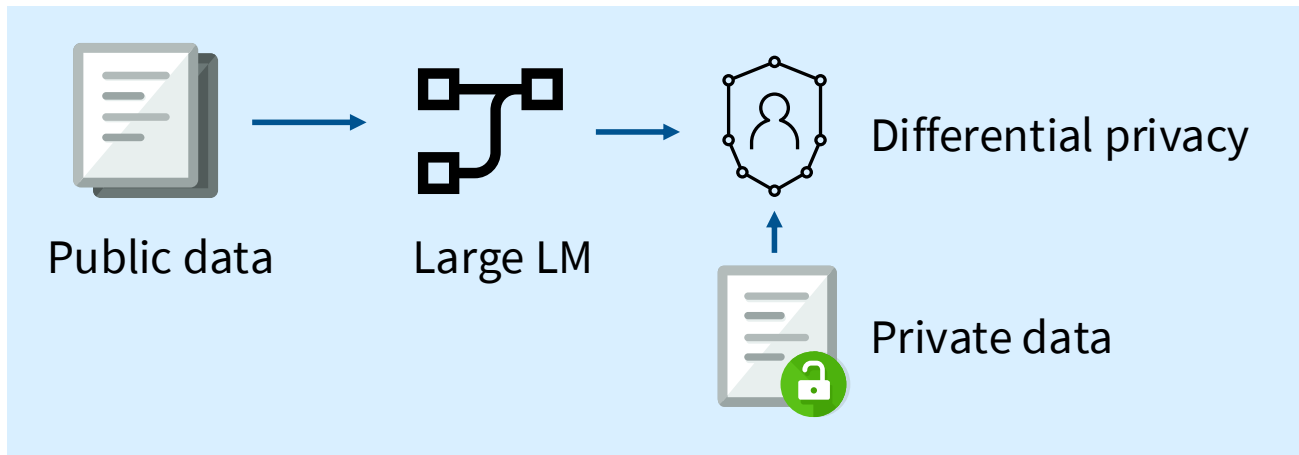
What happens when you don't tune a hyperparameter?

Let's go through a simple case study.. Building a “differentially private” language model

Open problem – large model size poses statistical + computational issues

Using a public language model to build a private downstream model

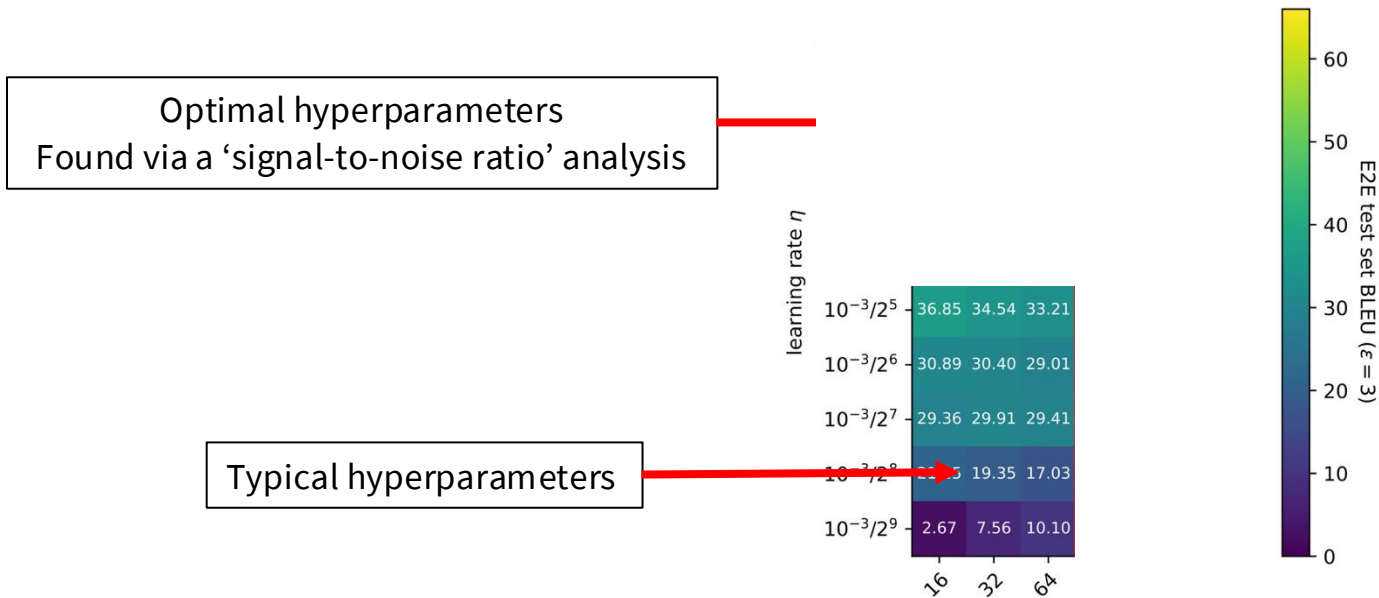
This is possible!



Doing the “obvious” thing (DP-SGD)

What people thought: The “obvious” thing of using DP-SGD doesn’t work

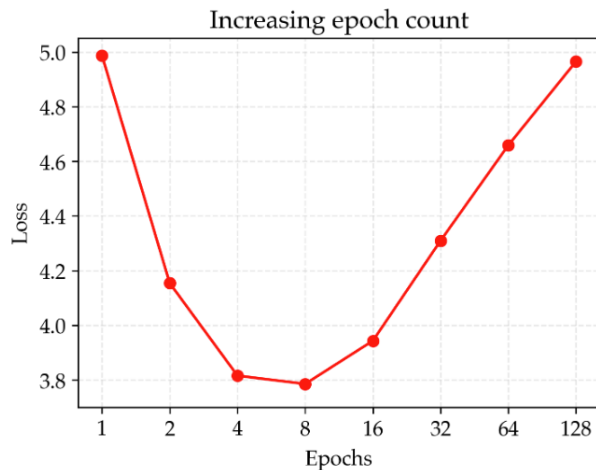
Actual phenomenon: People just didn’t tune the hyperparameters enough



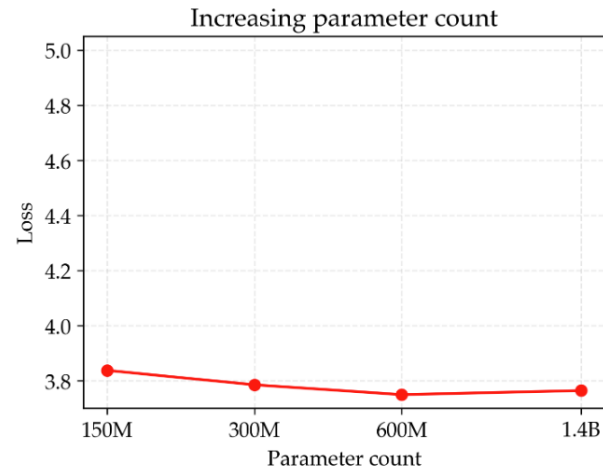
‘Naive’ choices were almost 100x off!

Case study 2: repeating data

Can repeating data give us better performance?



Tuned H	1	8	128
Learning rate	1e-3	1e-3	3e-3

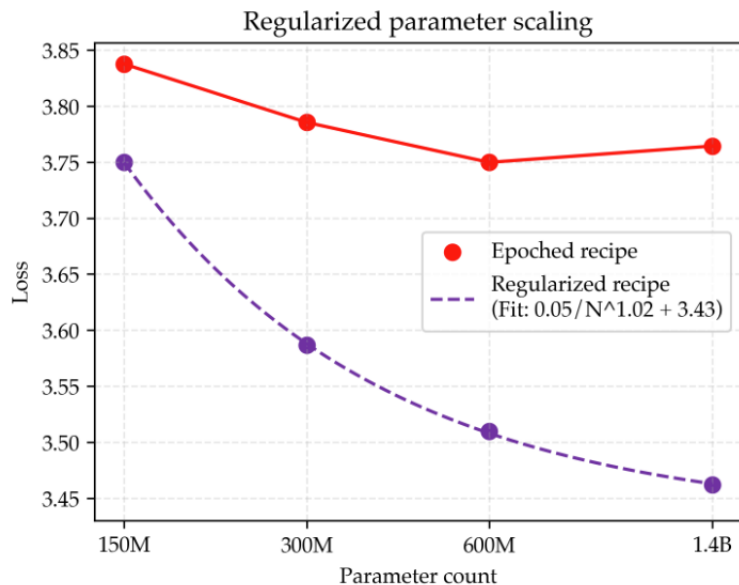


Tuned H	150M	300M	600M	1.4B
Learning rate	3e-3	1e-3	1e-3	3e-4
Epoch count	8	8	4	4

We get *no benefits* past a certain point (600M params w/ 200M dataset) even with infinite compute

Case study 2: repeating data

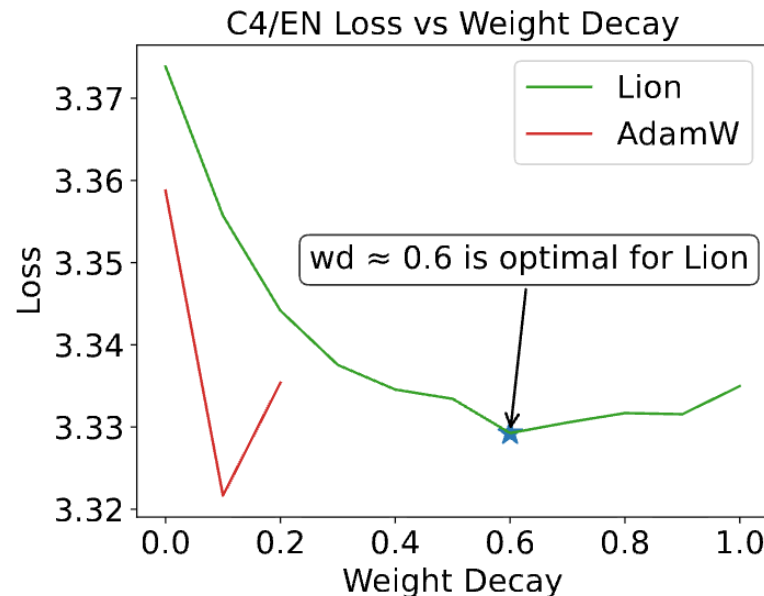
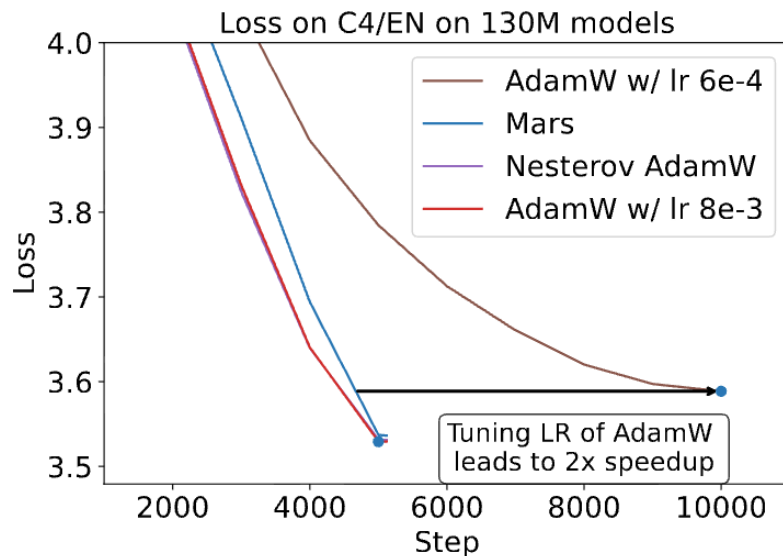
With optimal (or well tuned) regularization, we get clean trends



Tuned H	150M	300M	600M	1.4B
Learning rate	3e-3	3e-3	1e-3	1e-3
Epoch count	16	16	8	8
Weight decay	0.8	1.6	3.2	3.2

Changes the performance by a lot, *and* changes the qualitative conclusions

Case Study 3: AdamW alternatives



Different optimizers can require different hyperparameters!
(and likely have different optimal hyperparameter scaling)

What hyperparameters should we consider?

From assignment 1: a non-comprehensive list.

Hyperparameter	Default	Notes
Optimizer		
learning_rate	0.003	peak learning rate
optimizer_name	adamw	adamw or sgd
beta1	0.9	Adam β_1 (momentum)
beta2	0.95	Adam β_2
lr_schedule	linear	linear, cos, constant, $\text{wsd} < \frac{\text{frac}}{\text{}} >$ (e.g. $\text{wsd} 0.2$: stable then linear decay over the last 20%)
warmup_percent	0.01	linear warmup, as a fraction of total steps
weight_decay	0.1	masked: no decay on embeddings, norms, or biases
grad_norm	1.0	gradient-clipping threshold; set to None to disable
Data		
num_train_sequences	600,000	training horizon; each sequence is 1024 tokens
num_epochs	1.0	fractional values allowed, > 1 repeats data
batch_size	64	sequences per optimizer step
context_length	1024	fixed by the preprocessed dataset
data_seed	42	shuffle order of training sequences
Model		
depth (d4-d18)	d8	built-in ladder: layers = depth, hidden = 64-depth, heads = hidden/64, MLP = $3.5 \times$ hidden, head_dim=64, vocab_size=4096, context length 1024. Custom LMConfigs can override these choices.
qk_norm	true	normalize queries and keys before attention dot products
tie_word_embeddings	false	share input and output embedding weights when true
model_seed	42	initialization randomness
dropout	0.0	shared attention and residual-branch dropout probability
Systems		
num_micro_batches	1	gradient accumulation chunks per optimizer step; usually only needed above batch size 256
deterministic	false	request deterministic PyTorch behavior when possible
precision	mp	mp, bf16, or fp32

This is a lot! It's very high dimensional and potentially hopeless if hyperparam $\rightarrow$ loss is an arbitrary function in 21 dims

Let's walk through a hyperparameter investigation together

Large scale scaling law study from StepFun

Predictable Scale: Part I, Step Law – Optimal Hyperparameter Scaling Law in Large Language Model Pre-training

Houyi Li*
StepFun, Fudan University

Wenzhen Zheng*
StepFun

Qiufeng Wang
StepFun

Hanshan Zhang
StepFun

Zili Wang
StepFun

Shijie Xuyang
StepFun, Fudan University

YuanTao Fan
StepFun

Zhenyu Ding
Xi'an Jiaotong University

Haoying Wang
Xi'an Jiaotong University

Ning Ding
Xi'an Jiaotong University

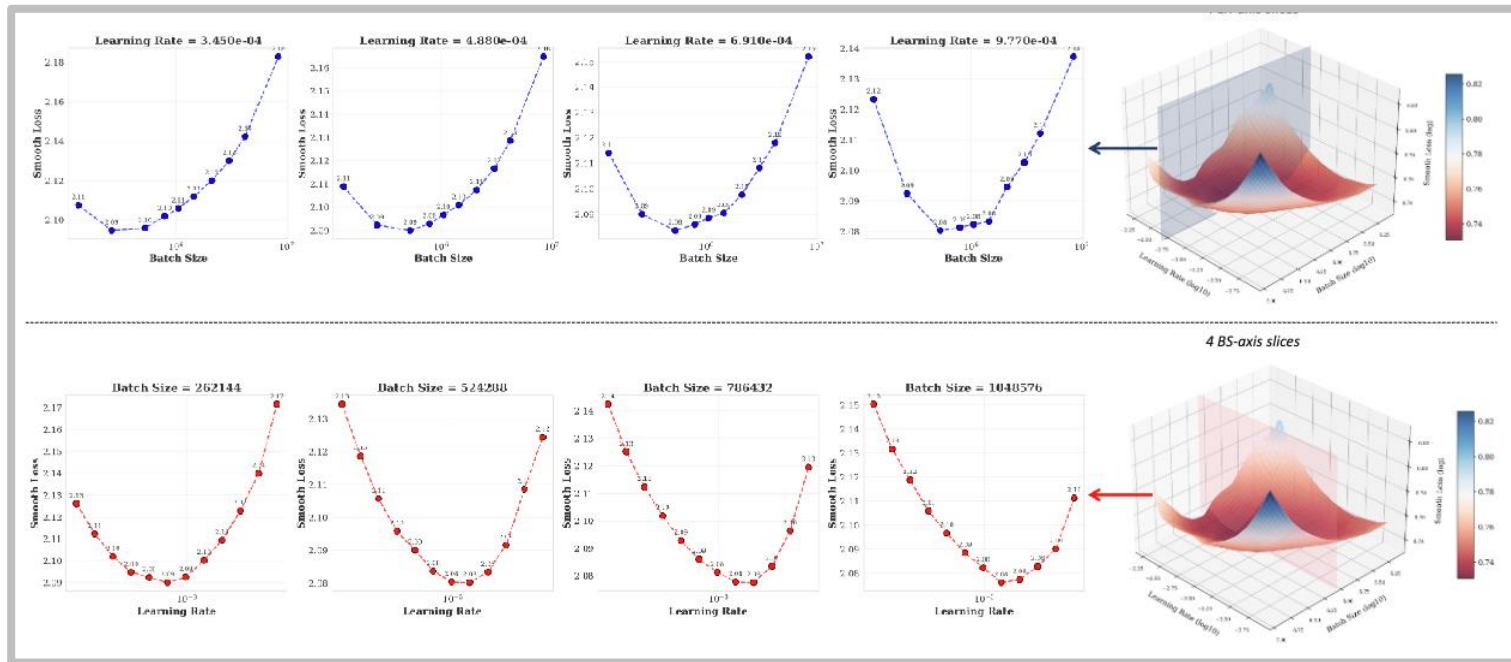
Shuigeng Zhou
Fudan University

Xiangyu Zhang
StepFun, Megvii Technology

Daxin Jiang
StepFun

Core question: how do we set LR / batch hyperparameters?

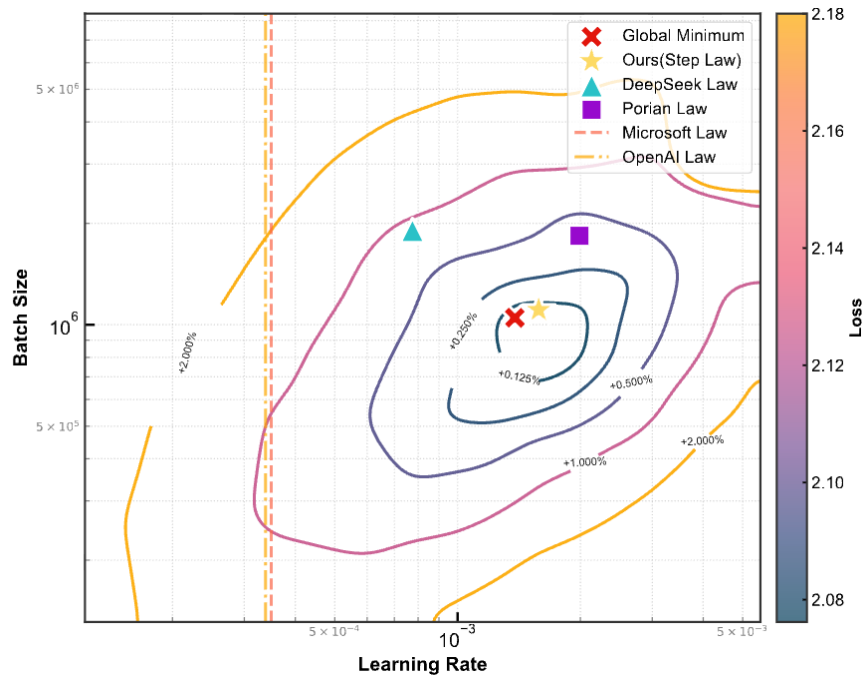
First step: how does performance change with LR and BS?



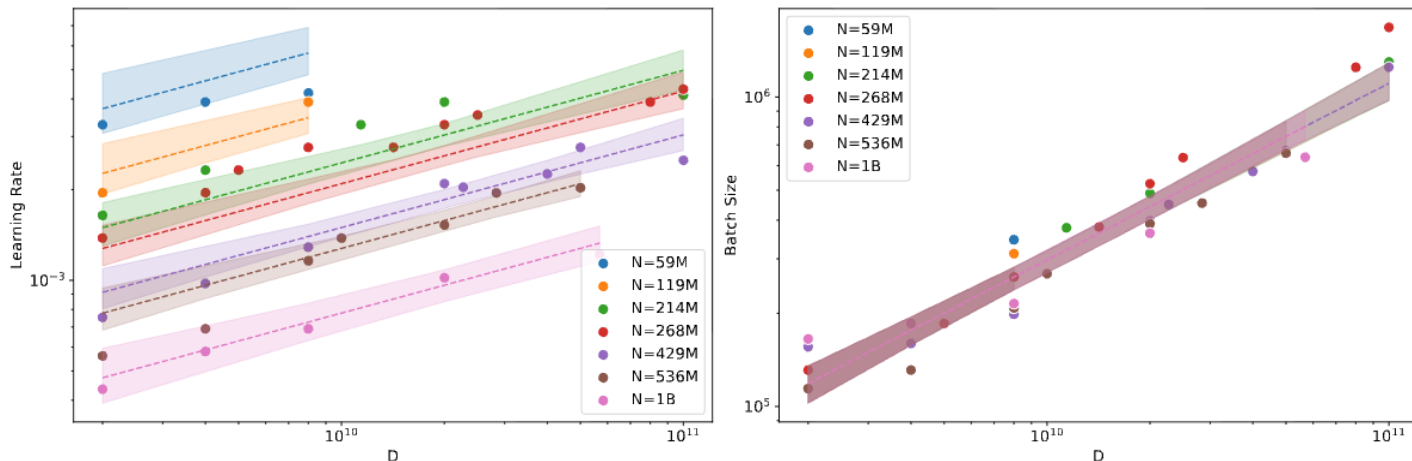
1. Using the same optimal LR isn't *too* bad (yay AdamW)
2. But the optimum does shift with batch size!
3. Nice, quadratic-ish shape in log-LR space.

A 2d version of this surface

Thanks to StepFun compute, we can see a ‘nice’ surface on how these two seem to relate



How does this change with model size / data size?

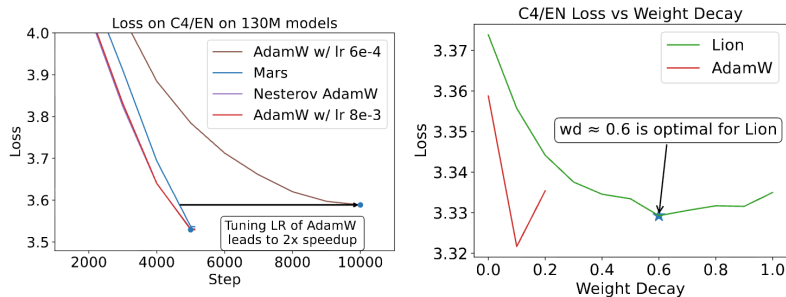


As we change the dataset size (D), the optimal batch size and learning rates both shift.
As we change the model size (N), the optimal LR also shifts!

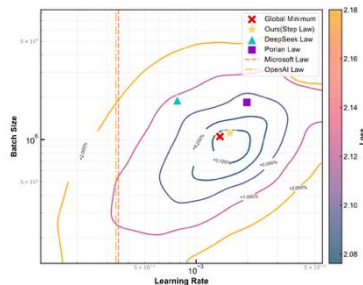
We need to always be vigilant in making sure our hyperparameters are (near) optimal!

Rough mental model of hyperparameters

Hyperparams need to be constantly re-tuned (esp for new methods and settings)



Have a good sense of what to tune, and how

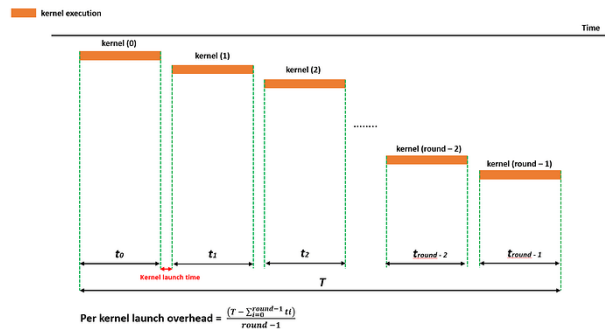


Be efficient with your compute! (hyperparameters are ultimately very trial-and-error)

One final trick: vmapping

To search over big hyperparameter grids, we want small models.

But! Small models are sometimes *very* inefficient due to kernel launch overheads



Is there something better we can do? **Yes!** A 'vmap' is generalized batching that's useful



Hyperparameters

The unglamorous, but very important part of empirical ML research.

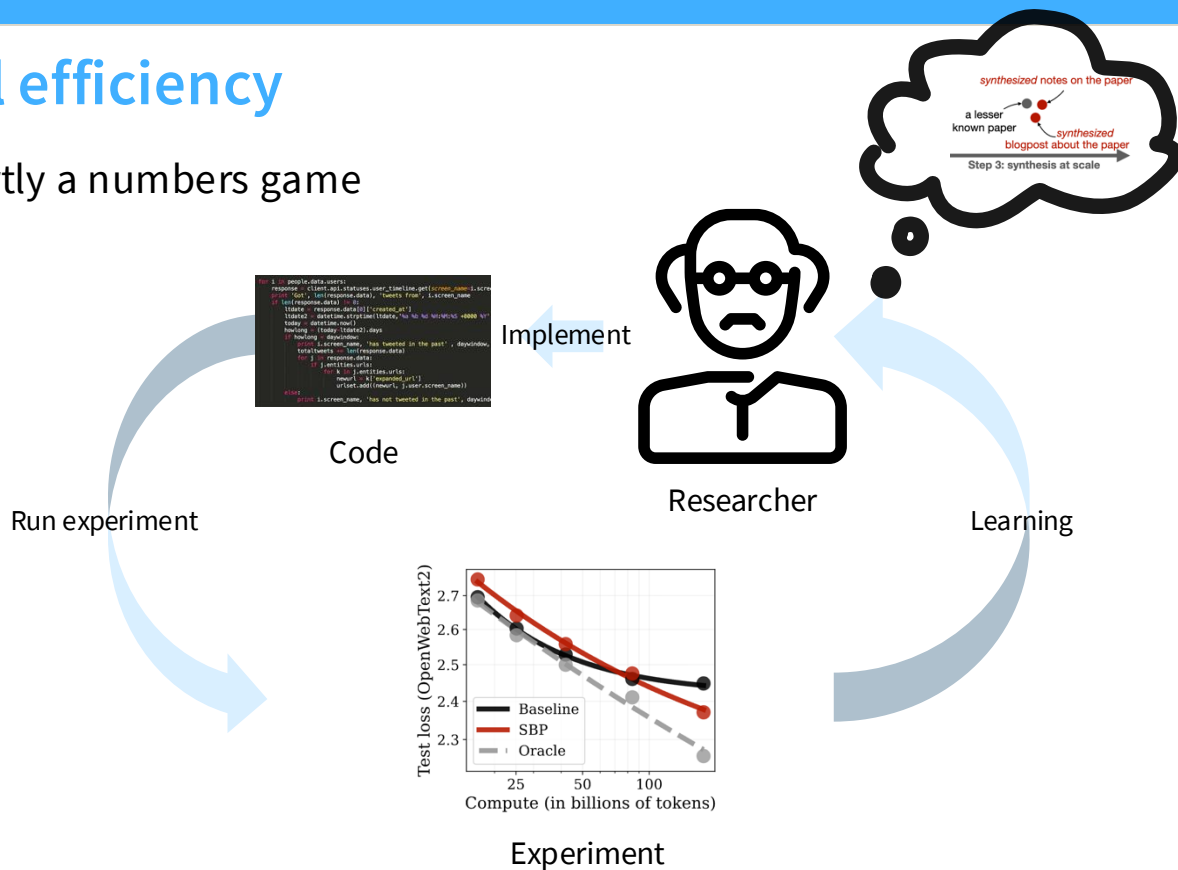


In theory its not hard ('just search') but practically complex

- Searching the 'right' variables and scales (takes experience)
 - Being rigorous about falsifying your own setup
- Picking the right scale to trade off number of experiments vs fidelity

Experimental efficiency

Deep learning is partly a numbers game



Your best shot at learning and optimizing systems is to run lots of experiments

Bottlenecks in empirical ML

There are bottlenecks to each of the 3 steps

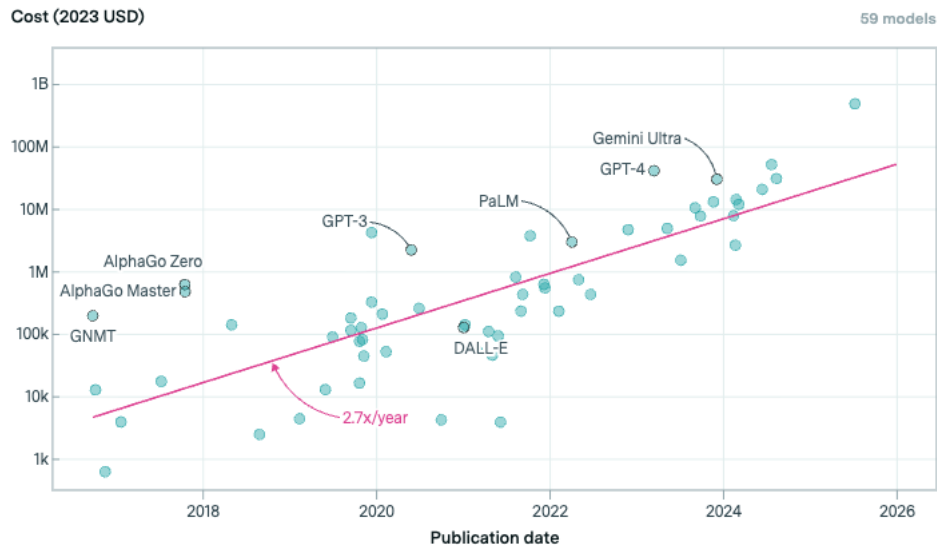
Learning: we need to think, absorb experiments (but maybe you're fast at this)

Implementation: getting easier all the time with agents

Running experiments: FLOPs bound. At larger labs, your experiments get bigger.

How can we run experiments more efficiently? Core answer: scaling laws

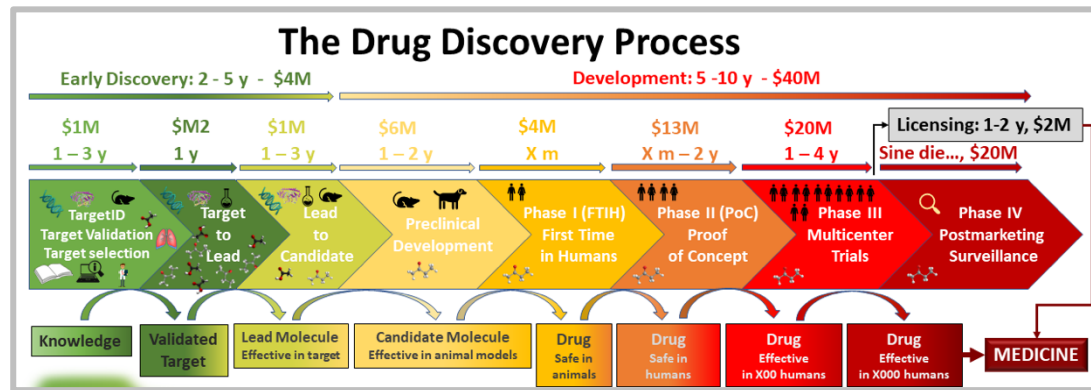
Dealing with expensive experiments in ML



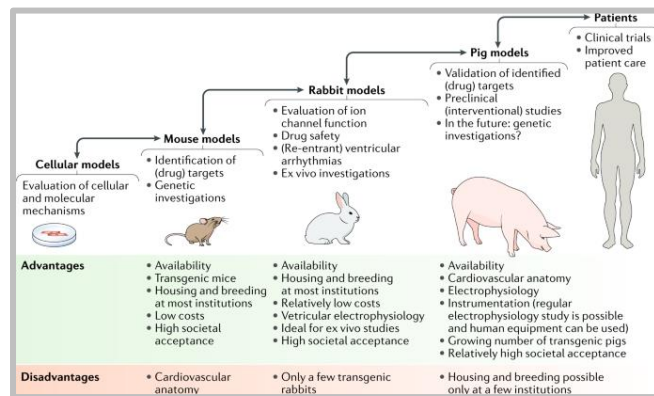
Especially in LLM land: experiments are getting more and more expensive! What do we do?

Cheap proxies to expensive realities

This is far from unique
biology has similar challenges

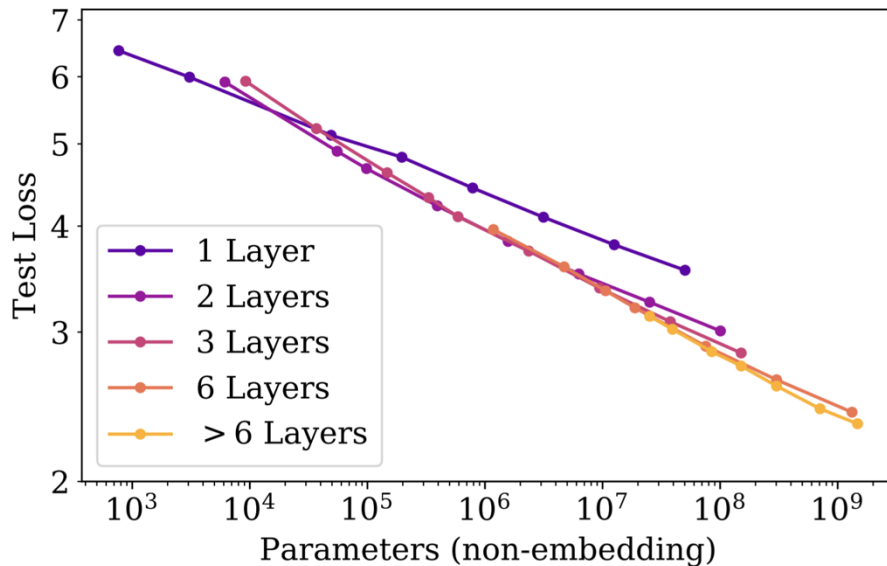


What we need
simple, model
systems and an
understanding of
their limits



Scaling laws as ‘cheap proxies’

Thankfully ML is easier to work with than biology (for now)



We can run experiments at various ‘scales’ (more realistic) and draw inferences
Surprising, and very strong regularities across compute scales!

Earliest (data) scaling law paper – 1993

Learning Curves: Asymptotic Values and Rate of Convergence

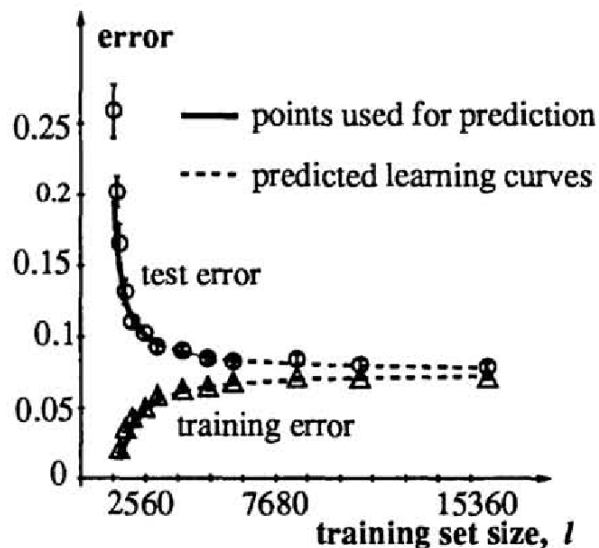
Corinna Cortes, L. D. Jackel, Sara A. Solla, Vladimir Vapnik,
and John S. Denker
AT&T Bell Laboratories
Holmdel, NJ 07733

Abstract

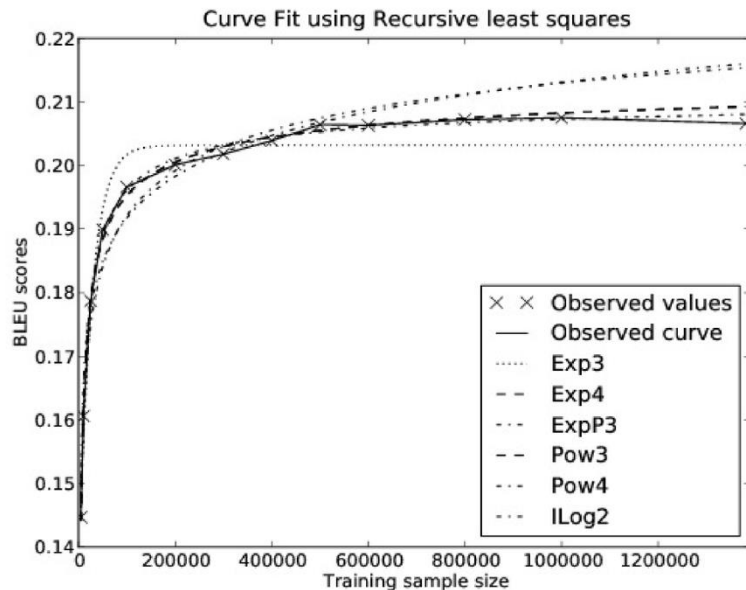
Training classifiers on large databases is computationally demanding. It is desirable to develop efficient procedures for a reliable prediction of a classifier's suitability for implementing a given task, so that resources can be assigned to the most promising candidates or freed for exploring new classifier candidates. We propose such a practical and principled predictive method. Practical because it avoids the costly procedure of training poor classifiers on the whole training set, and principled because of its theoretical foundation. The effectiveness of the proposed procedure is demonstrated for both single- and multi-layer networks.

A typical example of learning curves is shown in Fig. 2. The test error is always larger than the training error, but asymptotically they reach a common value, a . We model the errors for large sizes of the training set as power-law decays to the

$$\mathcal{E}_{\text{test}} = a + \frac{b}{l^\alpha} \quad \text{and} \quad \mathcal{E}_{\text{train}} = a - \frac{c}{l^\beta}$$



Early history of scaling laws – data scaling



Model	Formula
Exp ₃	$y = c - e^{-ax+b}$
Exp ₄	$y = c - e^{-ax^\alpha+b}$
ExpP ₃	$y = c - e^{(x-b)^\alpha}$
Pow ₃	$y = c - ax^{-\alpha}$
Pow ₄	$y = c - (-ax + b)^{-\alpha}$
LLog ₂	$y = c - (a/\log x)$

Table 1: Curve families.

Early tests of functional forms

Kolachina et al 2012 – power law relation between data and downstream performance

Hestness et al 2017

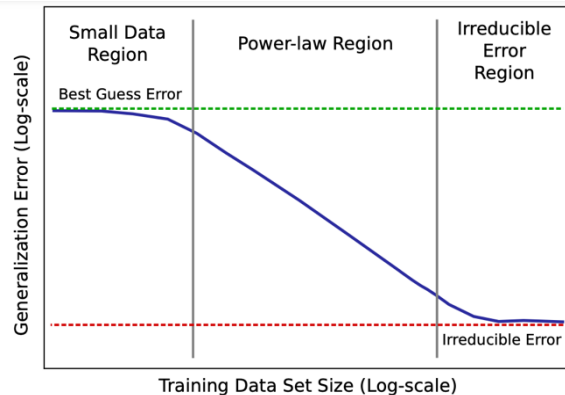
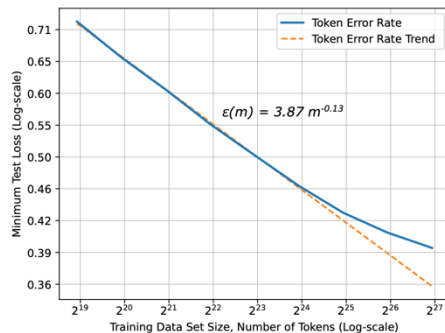
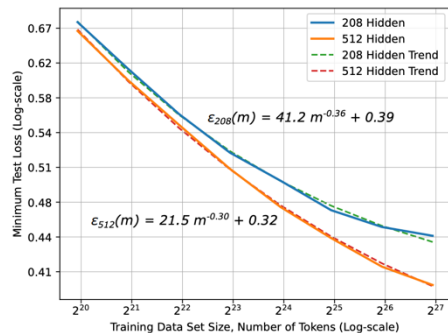


Figure 1: Neural machine translation learning curves. Left: the learning curves for separate models follow $\epsilon(m) = \alpha m^{\beta_g} + \gamma$. Right: composite learning curve of best-fit model at each data set size.

Earliest ‘large scale neural’ scaling work: Hestness 2017

Predictable scaling on many tasks (MT, LM, Speech) and hypothesized scaling shape.

One important use of scaling laws

Q: Do we need more data or bigger models?

Clearly, lots of data is wasted on small models

Joint data-model scaling laws describe how the two relate

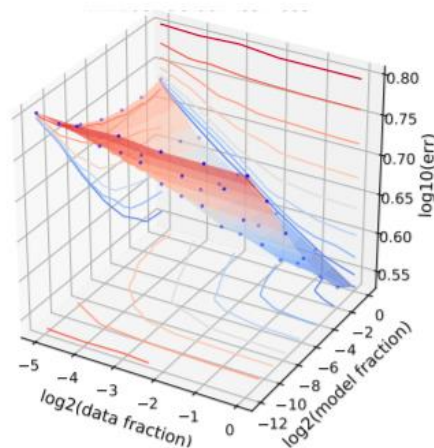
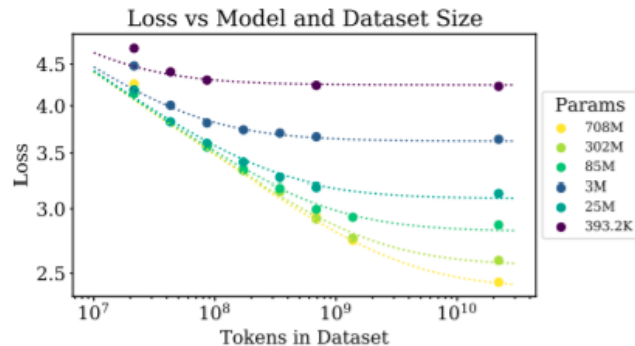
From Rosenfeld+ 2020,

$$Error = n^{-\alpha} + m^{-\beta} + C$$

From Kaplan+ 2020

$$Error = [m^{-\alpha} + n^{-1}]^{\beta}$$

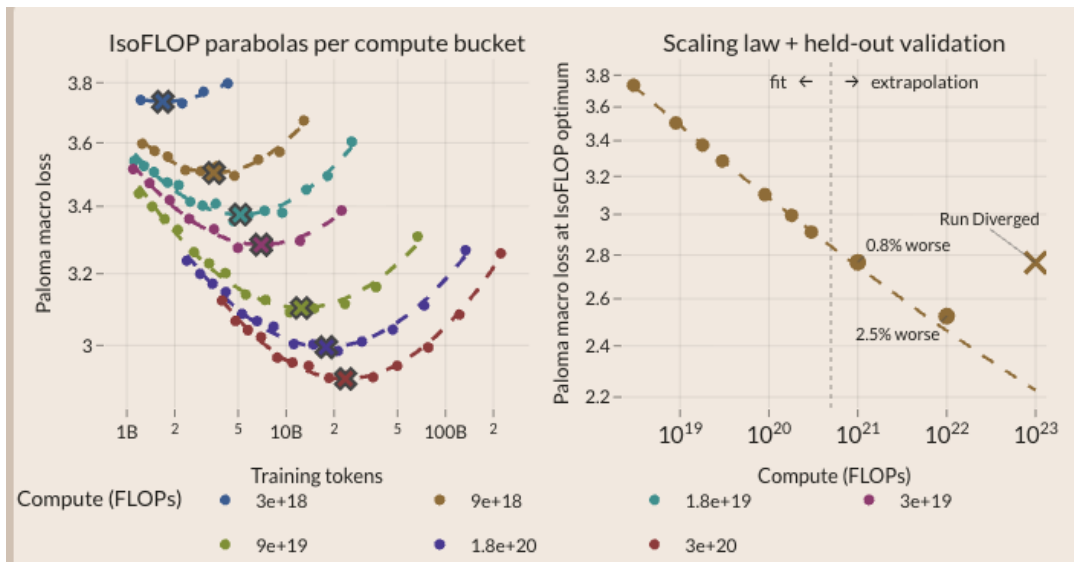
Provides surprisingly good fits to model-data joint error.



(a) Wiki103 error (cross entropy) landscape.

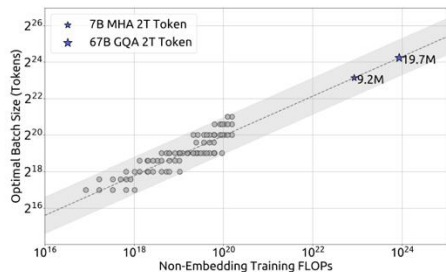
Pitfalls with scaling laws

Scaling laws don't come for free! *Tons of subtleties*

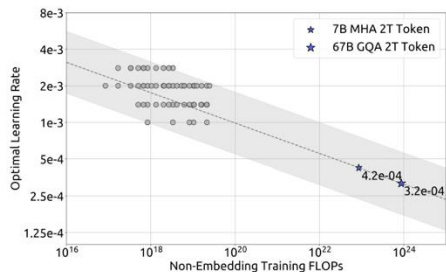


What are all the things that could fail when scaling?

Scaling vs hyperparameters



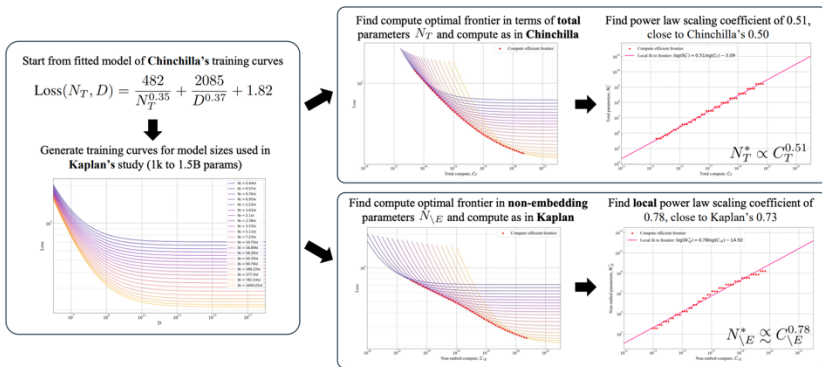
(a) Batch size scaling curve



(b) Learning rate scaling curve

Optimal hypers
change with scale!

Parameter / flops accounting



Which parameters count?
Is an embedding lookup a FLOP?

Teaser: scaling hyperparameters

Key trick: find scaling rules for optimal hyperparameters

Name	Data Recipe	Model Sparsity	Learning Rate	Batch Size	Relative Error
OpenAI Law [20]	✗	✗	$3.239 * 10^{-3} + -1.395 * 10^{-4} \log(N)$	$2e18 \mathcal{L}^{-4.76190}$	9.51‰
Microsoft Law [2]	✗	✗	$1.3192e^{-5} N^{-0.23} D^{-0.32}$	-	9.25‰
DeepSeek Law [6]	✗	✗	$0.3188 C^{-0.1250}$	$0.2920 C^{0.3271}$	9.26‰
Porian Law [26]	✗	✗	$3.7 N^{-0.36}$	$0.7576 N^{0.703}$	3.71‰
MiniCPM Law [18]	✗	✗	-	$\frac{2e18}{L^{6.24}}$	-
MeiTuan Law [36]	✗	✓	$\lambda \mathcal{L}^{-\alpha}$	$\lambda_B \mathcal{L}^{-\alpha_B^{-1}}$	-
Ours (Step Law)	✓	✓	$1.79 N^{-0.713} D^{0.307}$	$0.58 D^{0.571}$	0.94‰

Or invariances (i.e. learning rate = $O(\text{sqrt}(\text{batch_size}))$)

This lets us dramatically simplify the search and scaling process!

Scaling recap

Scaling is critical to modern empirical ML! Data and model scale impact perf

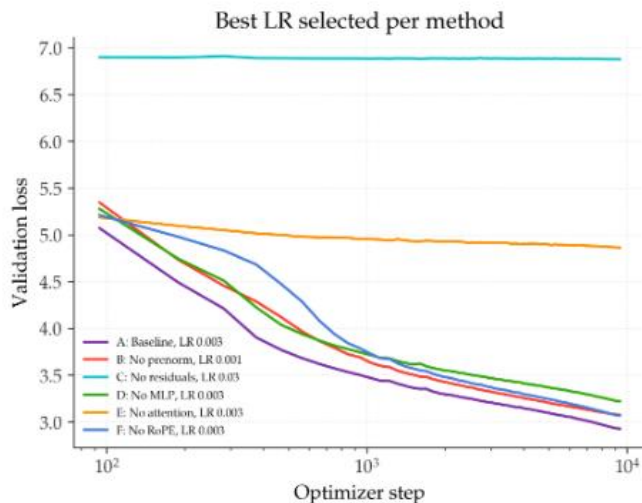
Predictability of scaling can be a very powerful tool in experiment efficiency

But also, scaling can be pretty tricky – it can look good until it's not

How do we know if methods are better or worse?

Is method A actually better than method B?

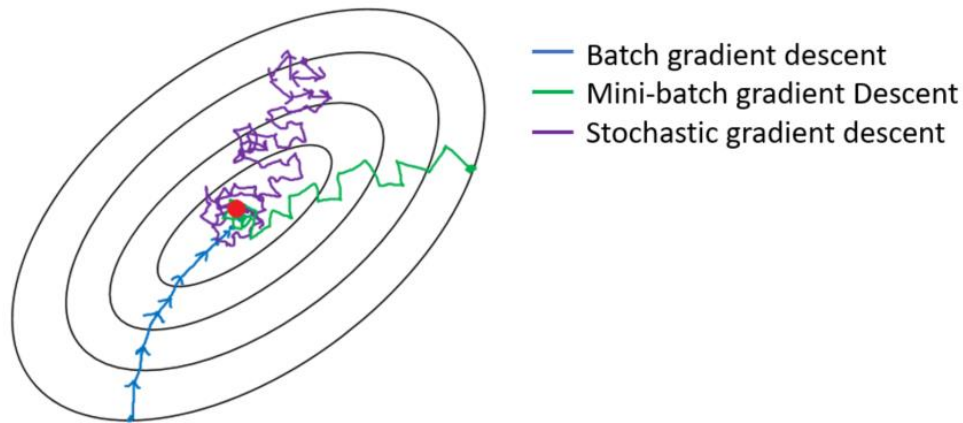
Variant	Intervention	Best LR	Final loss	Gap to A
A	Baseline	0.003	2.9267	+0.0000
B	No prenorm	0.001	3.0766	+0.1498
C	No residuals	0.03	6.8745	+3.9478
D	No MLP	0.003	3.2199	+0.2931
E	No attention	0.003	4.8641	+1.9373
F	No RoPE	0.003	3.0671	+0.1403



Many layers to this question, such as if methods are tuned right.
But let's start with an even more basic one: is this statistically significant?

Modern deep learning is a random process

We train most models using SGD (and friends, like Adam), often for compute reasons.



(Minibatch) SGD operates by gradient descent on *randomly* sampled examples' losses.
This is, of course, stochastic (from the random sampling)

Modern deep learning is a random process

We also initialize neural networks randomly. For D_{in} input and D_{out} output dims

Glorot / Xavier Initialization

$$W \sim \text{Unif}\left(-\sqrt{\frac{6}{D_{in} + D_{out}}}, \sqrt{\frac{6}{D_{in} + D_{out}}}\right)$$

He Initialization

$$W \sim \text{N}\left(0, \sqrt{\frac{2}{D_{in} + D_{out}}}\right)$$



(and more. The point is to do symmetry breaking)

Sources of randomness

Data choice and order

For all stochastic gradient methods, major source of randomness

Initialization

Necessary for sane functioning of neural nets.

Stochastic regularization

Not something we'll do *too* much with here, since it's uncommon in LM land

How much does each source contribute?

Decomposing sources of randomness

ON THE VARIANCE OF NEURAL NETWORK TRAINING
WITH RESPECT TO TEST SETS AND DISTRIBUTIONS

Keller Jordan*
Independent researcher

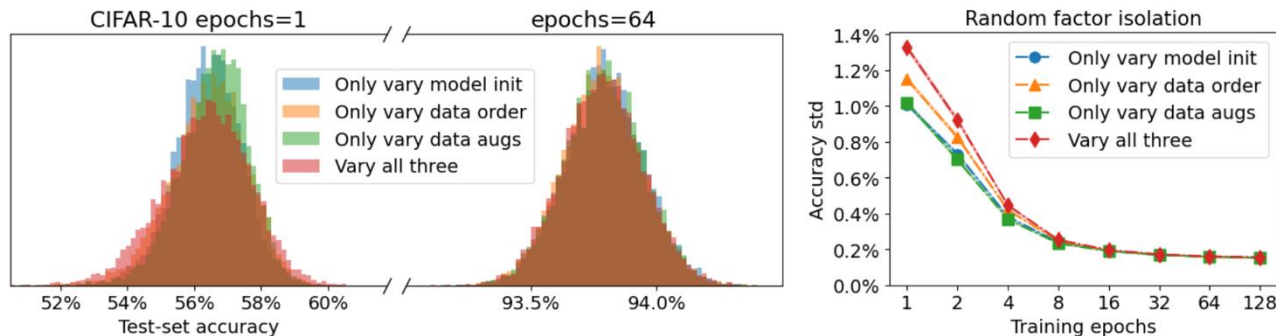
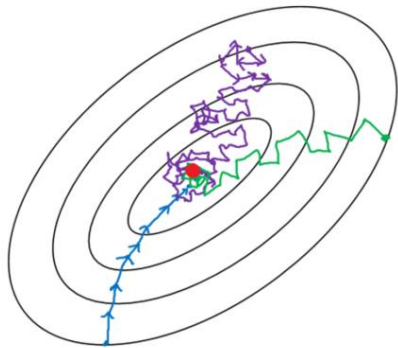


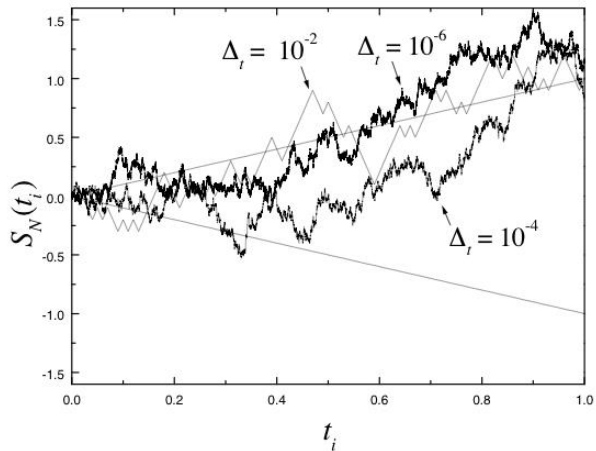
Figure 10: **One source of stochasticity suffices for long training.** When training for only 1 epoch, varying all three sources of randomness induces a standard deviation of 1.33% in test-set accuracy between runs, while each source alone induces 25-40% less variance. But when training for 64 epochs, varying any one source induces as much variance as all three together. Each distribution corresponds to 4,000 runs of training.

In CIFAR, randomness sources are *not* additive. Any single source leads to similar variance

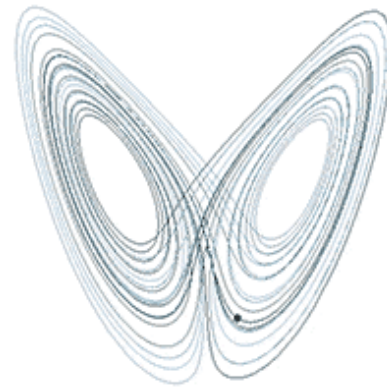
Mental models of deep learning randomness



SGD is like GD + noise



SGD is like Brownian Motion



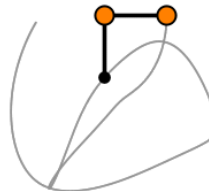
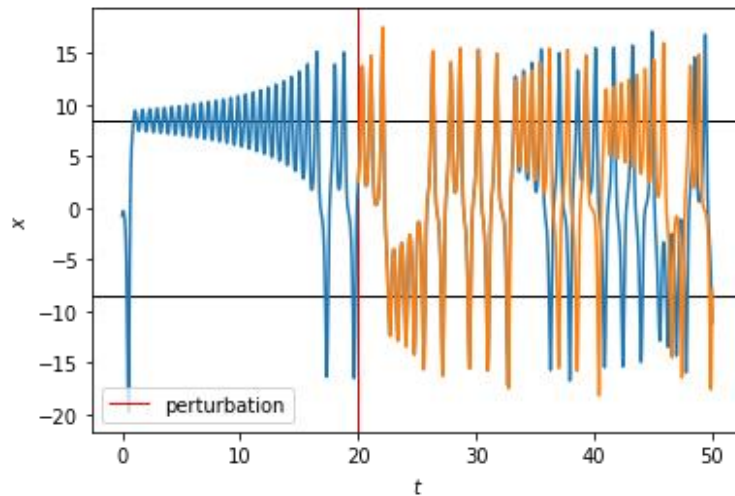
SGD is like a chaotic system

“Small perturbations keep you close”

“Small perturbations take you far”

Chaotic systems

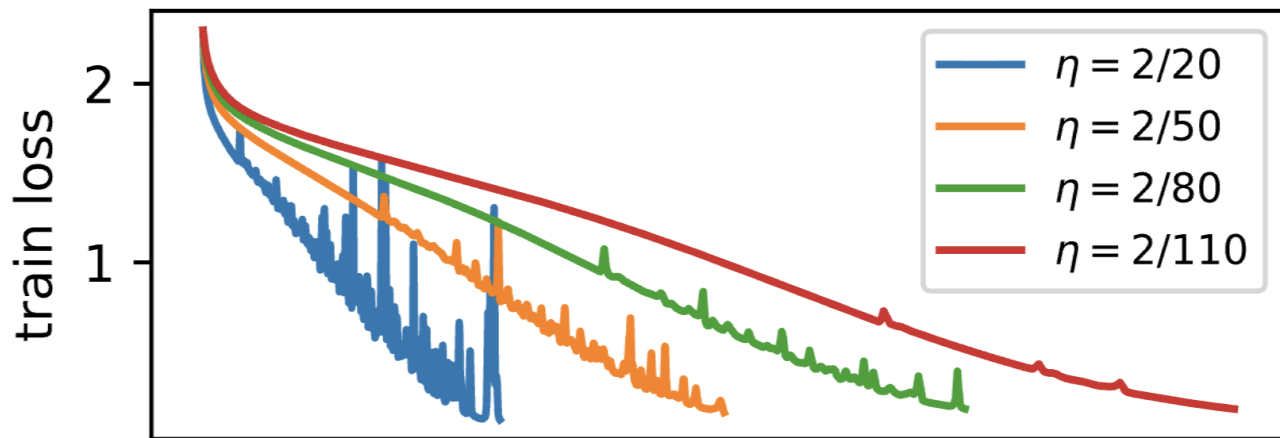
Infinitesimal perturbations can lead to macroscopic divergence



Examples – the weather, human behavior, etc .. But also, training deep nets?

Slight spoilers: ‘stochastic’ behavior in deterministic training

Fully-connected net on CIFAR-10 5k subset



A full batch gradient descent run on CIFAR.

There should be no stochasticity at all! Why do we see ‘stochastic’ spikes?

Know your stochasticity!

Many of the best architecture works in the past had careful treatment of randomness

GLU Variants Improve Transformer

Noam Shazeer
Google
noam@google.com

February 14, 2020

Training Steps	65,536	524,288
$\text{FFN}_{\text{ReLU}}(\text{baseline})$	1.997 (0.005)	1.677
FFN_{GELU}	1.983 (0.005)	1.679
$\text{FFN}_{\text{Swish}}$	1.994 (0.003)	1.683
FFN_{GLU}	1.982 (0.006)	1.663
$\text{FFN}_{\text{Bilinear}}$	1.960 (0.005)	1.648
$\text{FFN}_{\text{GEGLU}}$	1.942 (0.004)	1.633
$\text{FFN}_{\text{SwiGLU}}$	1.944 (0.010)	1.636
FFN_{ReLU}	1.953 (0.003)	1.645

Do Transformer Modifications Transfer Across Implementations and Applications?

Sharan Narang* Hyung Won Chung Yi Tay William Fedus
Thibault Feyry† Michael Matena† Karishma Malkan† Noah Fiedel
Noam Shazeer Zhenzhong Lan† Yanqi Zhou Wei Li
Nan Ding Jake Marcus Adam Roberts Colin Raffel

Google Research

Model	Params	Ops	Step/s	Early loss	Final loss	SGLUE
Vanilla Transformer	223M	11.1T	3.50	2.182 ± 0.005	1.838	70.97
GeLU	223M	11.1T	3.58	2.179 ± 0.003	1.838	73.67
Swish	223M	11.1T	3.62	2.186 ± 0.003	1.847	72.03
ELU	223M	11.1T	3.56	2.270 ± 0.007	1.932	65.86
GLU	223M	11.1T	3.59	2.174 ± 0.003	1.814	67.86
GeGLU	223M	11.1T	3.55	2.130 ± 0.006	1.792	74.86
ReGLU	223M	11.1T	3.57	2.145 ± 0.004	1.803	73.40
SeLU	223M	11.1T	3.55	2.315 ± 0.004	1.948	66.13
SwiGLU	223M	11.1T	3.53	2.127 ± 0.003	1.789	74.21
LiGLU	223M	11.1T	3.59	2.149 ± 0.005	1.798	73.97
Sigmoid	223M	11.1T	3.63	2.291 ± 0.019	1.867	71.27
Softplus	223M	11.1T	3.47	2.207 ± 0.011	1.850	70.86
RMS Norm	223M	11.1T	3.68	2.167 ± 0.008	1.821	73.73
Rezero	223M	11.1T	3.51	2.262 ± 0.003	1.939	57.21
Rezero + LayerNorm	223M	11.1T	3.26	2.223 ± 0.006	1.858	68.16
Rezero + RMS Norm	223M	11.1T	3.34	2.221 ± 0.009	1.875	65.14
Fixup	223M	11.1T	2.95	2.382 ± 0.012	2.067	55.71
24 layers, $d_{\text{ff}} = 1536$, $H = 6$	224M	11.1T	3.33	2.200 ± 0.007	1.843	72.55
18 layers, $d_{\text{ff}} = 2048$, $H = 8$	223M	11.1T	3.38	2.185 ± 0.005	1.831	74.74
8 layers, $d_{\text{ff}} = 4608$, $H = 18$	223M	11.1T	3.69	2.190 ± 0.005	1.847	72.17
6 layers, $d_{\text{ff}} = 6144$, $H = 24$	223M	11.1T	3.70	2.201 ± 0.010	1.857	70.28

Minor tip: paired significance tests

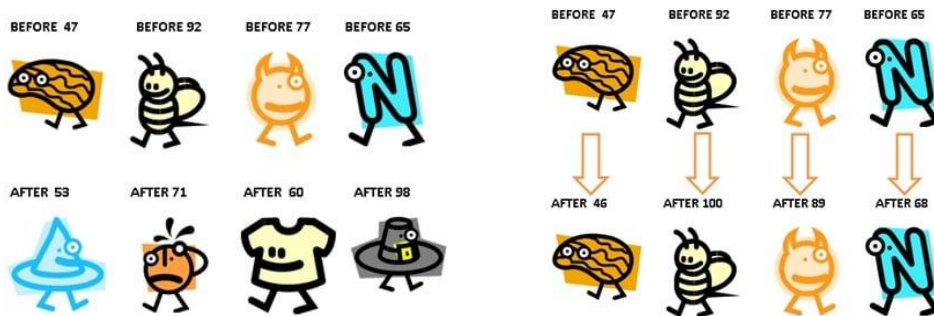
Stats 101 reminder: Usually, we generate standard errors ($\text{std dev}/\sqrt{n}$) and $1.96 \times$ this standard error is the 95% confidence interval.

You have two methods, and the standard errors overlap. Is the difference significant?

Often yes, in machine learning..

Key observation:

A lot of the variation in loss is variance in problem difficulty, not variability in training



Takeaway: Be vigilant and know your randomness, but also things are often ok

Recap: stochasticity

Treat **all** of deep learning as semi-stochastic

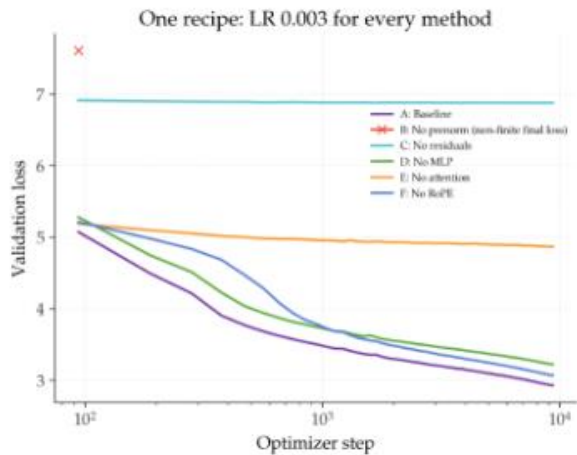
Know the run-to-run variation in your domain

Know how training and optimization amplifies (or damps) randomness

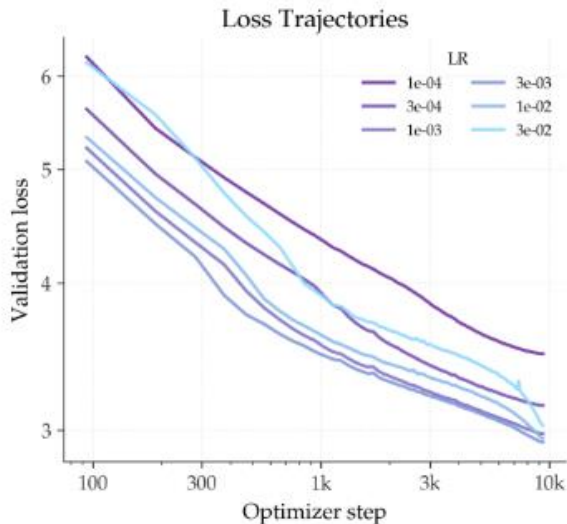
(But often, this is not your biggest problem)

What do we do when things don't work?

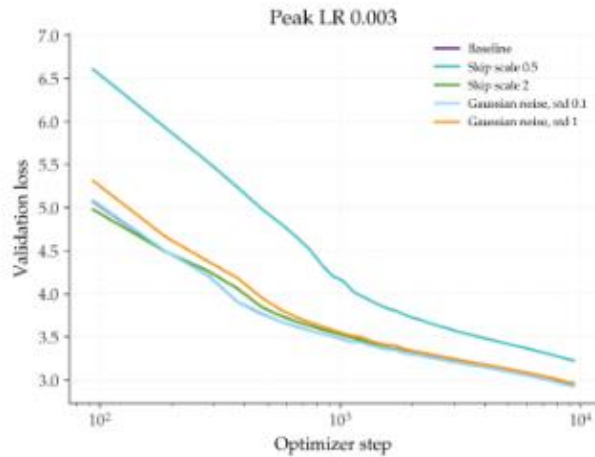
Training blows up



Training stops



It's less good than I thought



We need 'debugging tools' to see what's going on inside the model

Diagnostics in deep learning

What do we 'watch' in deep learning?

External observables

- Loss curves
- Model predictions (we won't deal with this)

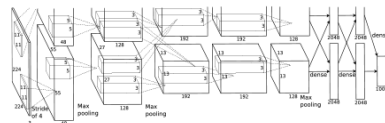
Internal observables

- Activations
- Gradients
- Parameters

When are internal diagnostics useful?

Isolating the failing component

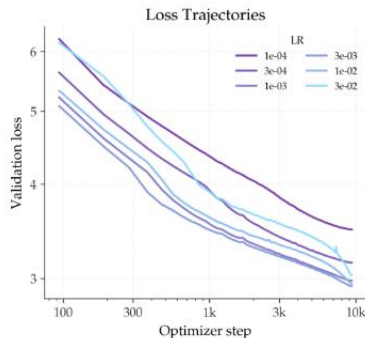
Forward: $f_W(x)$ (Activations)



“espresso”
+ loss

$\nabla f_W(x)$ Backward: learning (Gradients / Cotangents)

Tracking dynamics,
temporal effects

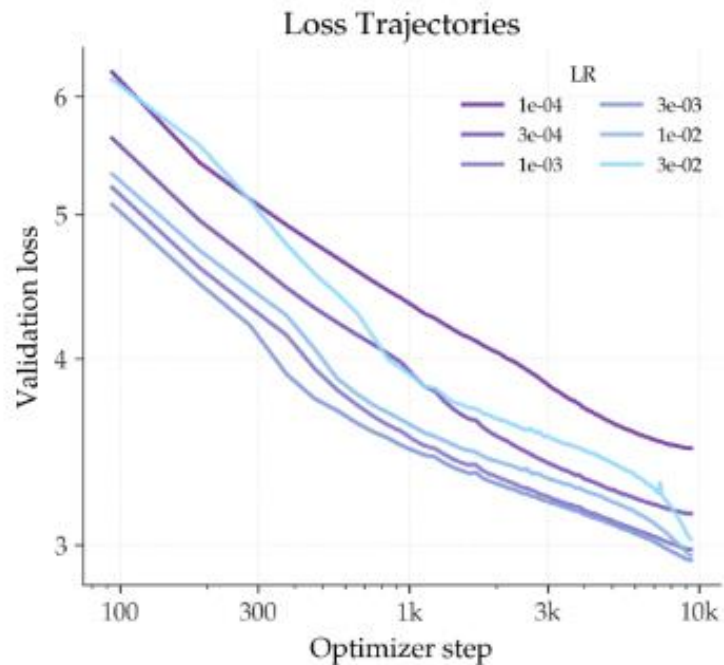


Parameters (and optimizer state)

Loss curve shapes

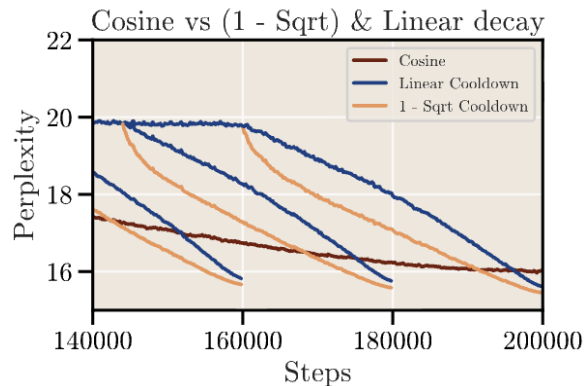
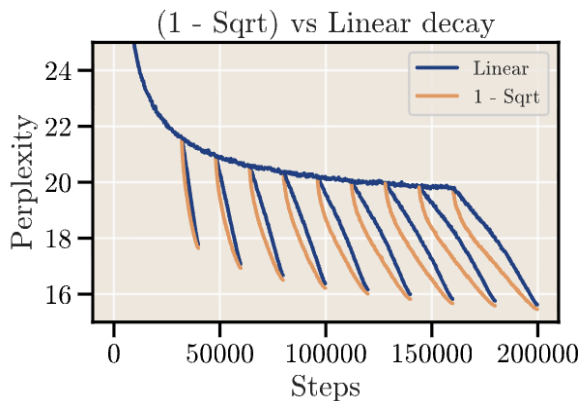
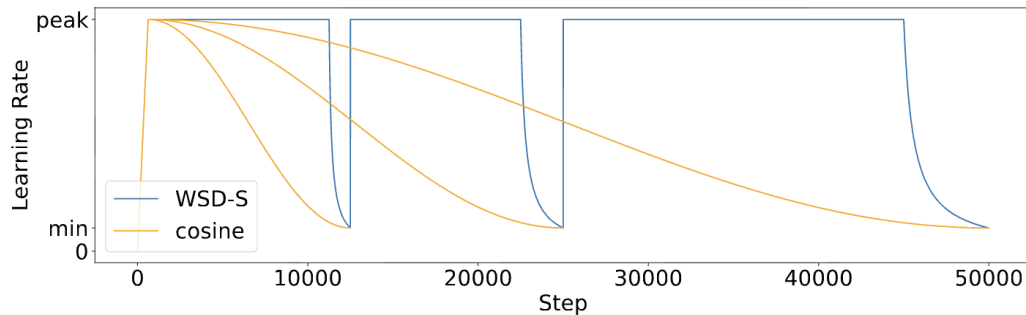
Learning rate scheduling / loss curves

Loss curves are one useful diagnostic!



What can change loss curves?

Learning rate schedules can leave clear signs on the loss curves



Activation / cotangent tracking

Xavier Glorot

DIRO, Université de Montréal, Montréal, Québec, Canada

Yoshua Bengio

What about internal monitoring? Activations and cotangents can inform signal propagation

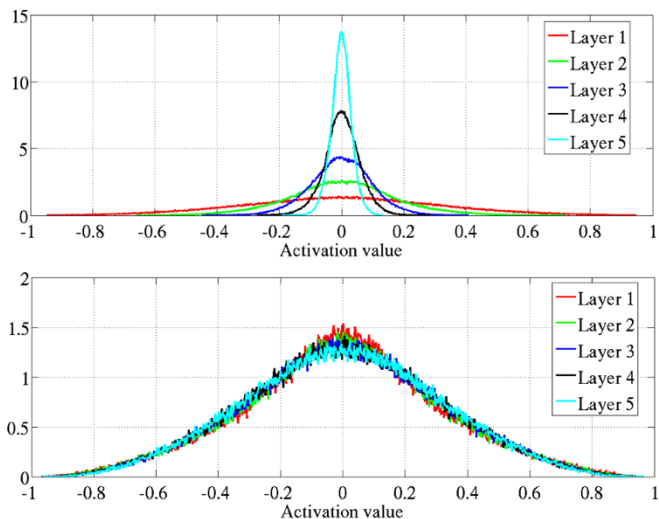


Figure 6: Activation values normalized histograms with hyperbolic tangent activation, with standard (top) vs normalized initialization (bottom). Top: 0-peak increases for higher layers.

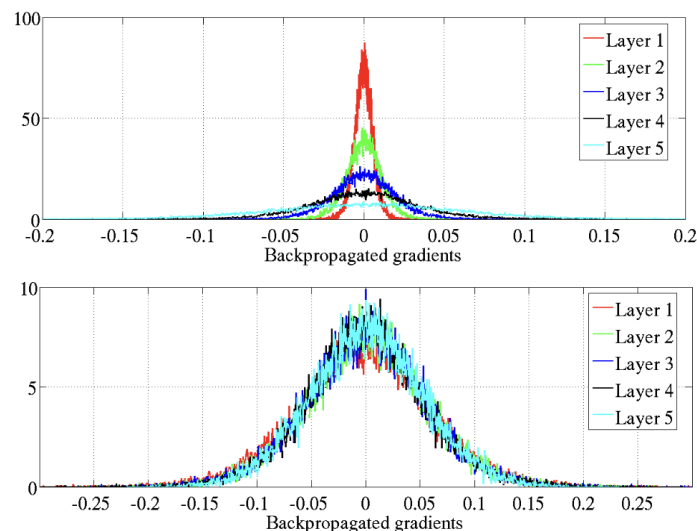
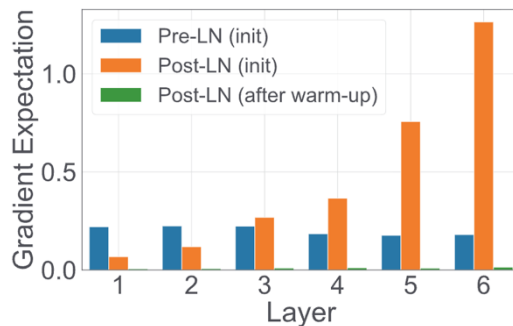
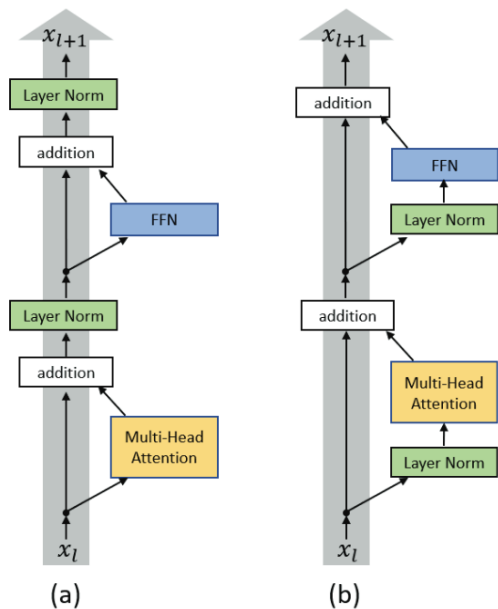


Figure 7: Back-propagated gradients normalized histograms with hyperbolic tangent activation, with standard (top) vs normalized (bottom) initialization. Top: 0-peak decreases for higher layers.

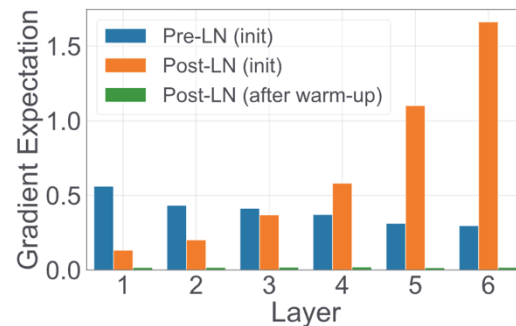
Pre-norm layer norms

Ruibin Xiong^{†*1,2} Yunchang Yang^{*3} Di He^{4,5} Kai Zheng⁴ Shuxin Zheng⁵ Chen Xing⁶ Huishuai Zhang⁵
Yanyan Lan^{1,2} Liwei Wang^{4,3} Tie-Yan Liu⁵

Similarly, gradient norms can tell us quite a bit about how (and if) models train well



(a) W^1 in the FFN sub-layers



(b) W^2 in the FFN sub-layers

QK norm

'QK norm' stabilizes training by making the attention and logit better behaved.

In the scaling ViT paper, we see that internal state tracking makes the mechanisms clear

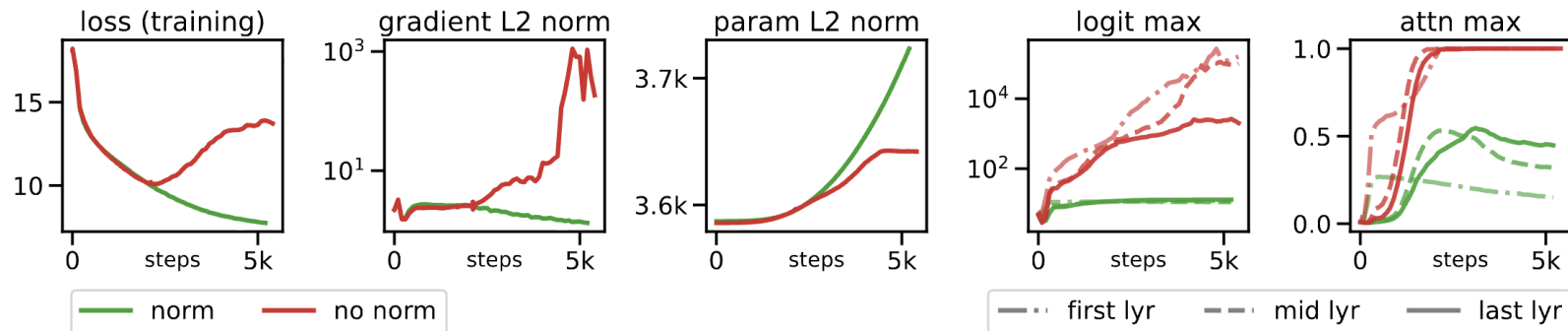


Figure 1: Effect of query/key normalization on an 8B parameter model.

Build intuition for ‘normal’ behaviors

Some key debugging tools that you should have

Have a good sense of what’s normal.

Are nonmontone / spikey losses normal? How much?

Should internal statistics grow or shrink? By how much?

Know your invariances.

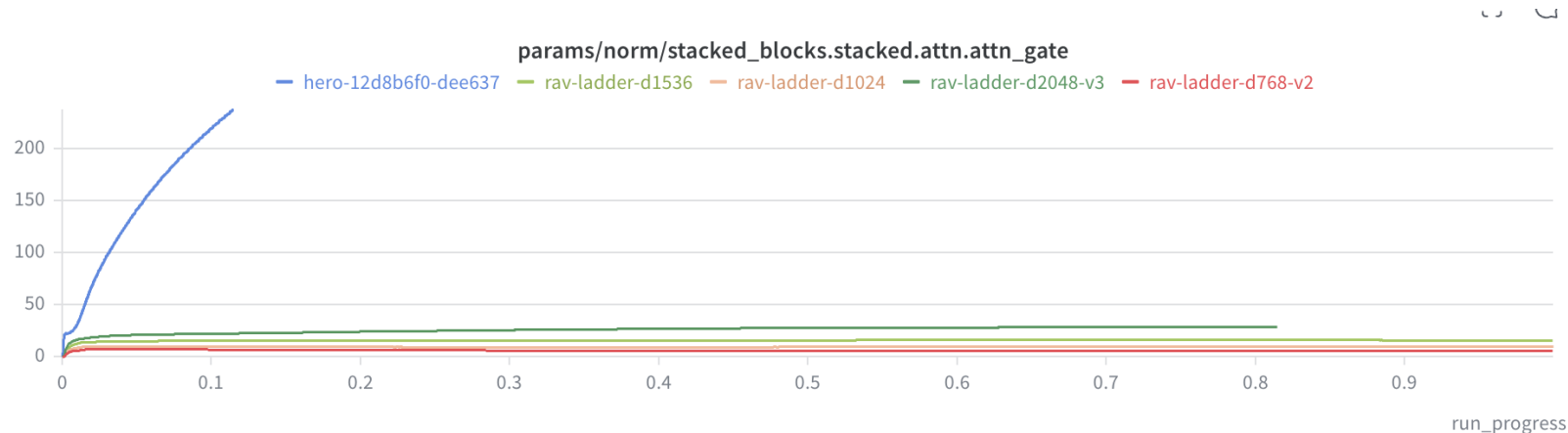
Certain factors can ‘blow up’ without affecting models
(a good example of this is models under layer norm)

Compare to ‘known good controls’

Have baselines, smaller scale runs, that you know work that you can compare to

“Live” example from the Marin training process

During training.. Strange phenomena compared to small scale models

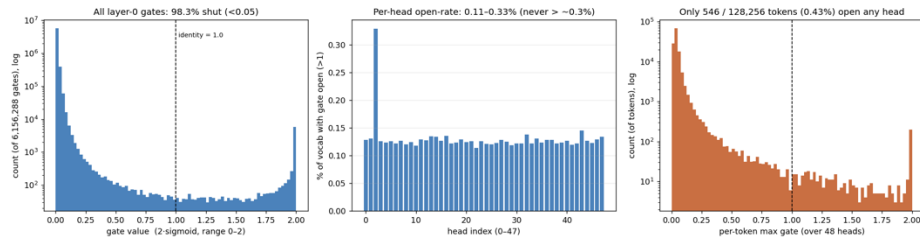


Pretty concerning shapes compared to the smaller scale ladders (colored lines)

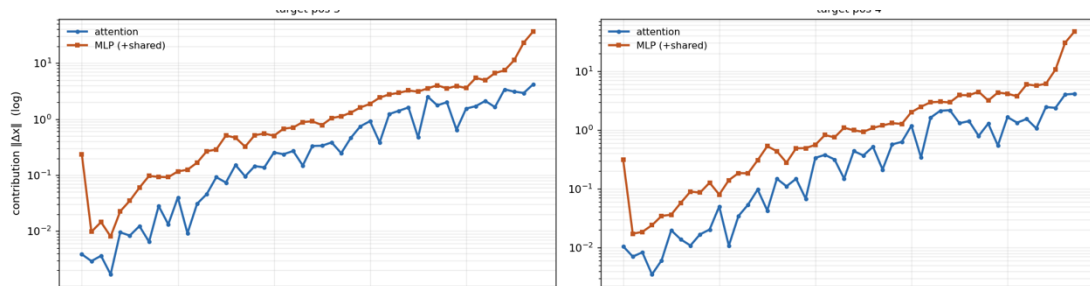
Very big parameter norms for 'gated attention' layers

Debugging steps.

Hero d6144 step-42k: layer-0 attention gate is —closed across the full 128k vocabulary (mean 0.013, median 0.007)



The early layers seem to have ‘attention gates’ that make activations small



Early layers seem to contribute very little to the residual stream

Solution: light weight decay to bring the norms back to ‘sane’ values.

Summary for diagnostics

Know what to track, and how these statistics look “normally”

Know when deviations on these metrics impact behavior (e.g. invariances)

Understand all the knobs you have at your disposal to intervene on the metrics

Other tips:

- Change one thing at a time
- Interpolate from known good (or bad) configurations

Overall summary

Experimental sciences require experiment skills! Think like a biologist or neuroscientist

Be paranoid about your system (hyperparams, stochasticity)

Be efficient, have good model systems (scaling)

Have controls – compare to known good and bad states (debugging)